

Lagerverwaltung - Dokumentation

POS-Projekt Lagerverwaltung (Java + Vaadin), Spengergasse.

Verwaltet werden Artikel in drei Kategorien: Elektronik, Lebensmittel und Kleidung. Jede Kategorie hat eigene Zusatzfelder und eine eigene Berechnung für den Bruttopreis, der Rest (ID, Name, Marke, Preis, Bestand, Favorit) ist gemeinsam in der abstrakten Klasse `Artikel`.

Aufbau

- `Artikel` (abstrakt) + Unterklassen `ElektronikArtikel`, `Lebensmittel`, `Kleidung`. Dazu das Enum `Groesse` für Kleidung (XS bis XL).
- `Lagerverwaltung` - hält die Artikelliste und macht die ganze fachliche Arbeit: Anlegen, Ändern, Löschen, Suchen, Filtern, Sortieren, Gruppieren, Auswertungen, Favoriten, und ein Verlaufs-Log, in dem jede Änderung mitgeschrieben wird.
- `DateiService` - liest/schreibt die Artikelliste als Datei, entweder im einfachen Textformat (Semikolon-getrennt) oder als CSV (Komma, mit Kopfzeile).
- `LagerService` - Spring-Service, der die GUI mit Daten versorgt. Lädt beim Start aus `data/lager.csv` und speichert nach jeder Änderung wieder dorthin.
- GUI mit drei Vaadin-Views (`ArtikellisteView`, `StatistikView`, `VerlaufView`) und gemeinsamem Rahmen (`MainLayout`).
- `LogEintrag` (ein Verlaufseintrag) und `UngueltigeEingabeException` als eigene geprüfte Exception für ungültige Eingaben.

Ungültige Eingaben (negative Preise, leere Namen, null-Werte, ...) lösen überall die gleiche `UngueltigeEingabeException` aus. Geprüft wird entweder im Konstruktor oder im jeweiligen Setter, beide rufen dieselben privaten `pruefe...`-Helfer auf.

Artikel-Klassen

`Artikel` hat zwei MwSt-Konstanten (`MWST_NORMAL` = 0.20, `MWST_REDUIERT` = 0.10), die sich die Subklassen teilen. `berechnePreis()` und `getKategorie()` sind abstrakt, jede Subklasse macht das auf ihre Art:

- **ElektronikArtikel**: Garantiemonate + optionale Batterie (`batterieTyp` ist null, wenn keine Batterie eingesetzt ist - es gibt also kein extra Boolean-Feld dafür). 20% MwSt.

- **Lebensmittel:** Haltbarkeitsdatum + Bio-Flag. `tageAbgelaufen()` rechnet die Differenz zum heutigen Datum aus, `istAbgelaufen()` und `istBaldAbgelaufen(tage)` bauen darauf auf. 10% MwSt (reduzierter Satz).
- **Kleidung:** Größe (Enum) + Material. `istGroesseKleinerGleich()` vergleicht über `ordinal()`, `istMaterialNatuerlich()` prüft eine kleine Liste an Naturfasern. 20% MwSt.

LagerVerwaltung

Die Klasse mit der meisten Logik. Kurzer Überblick, was es alles gibt:

- CRUD: `hinzufuegen`, `entfernen`, `aendern` (ersetzt einen Artikel komplett über die ID), `suchen` (nach Name, case-insensitive).
- Filter: nach Preisbereich, Mindestbestand, Marke, sowie `filtern(...)` als Kombi-Filter über Kategorie + Marke + Preisbereich gleichzeitig (leere/null Kriterien werden ignoriert).
- Sortierung: nach Name, Preis, Bestand - arbeitet jeweils auf einer Kopie der Liste, das Original bleibt unverändert.
- Auswertungen: Gesamtwert, Durchschnittspreis, Gesamtbestand, teuerster/günstigster Artikel, außerdem typgenau das teuerste Elektronikgerät und das teuerste Kleidungsstück.
- Gruppierung nach Kategorie (TreeMap, damit die Reihenfolge alphabetisch ist).
- `filterUndSortierenNachPreis(minPreis)` als Beispiel für eine Methode, die mehrere Schritte kombiniert (filtern + sortieren in einem Aufruf).
- Favoriten: markieren/demarkieren, dazu ein Filter nur für Favoriten.
- Verlaufs-Log: jede verändernde Aktion (hinzufügen/ändern/entfernen/Favorit) erzeugt einen `LogEintrag`. `getVerlauf()` gibt eine Kopie davon zurück.
- Datei-Operationen delegieren an `DateiService`, fangen aber `IOException` ab und wandeln sie in `UngueltigeEingabeException` um - der Aufrufer muss sich nur mit einer Exception-Art beschäftigen.

DateiService

Liest/schreibt beide Formate mit demselben Spaltenaufbau:

Typ	ID	Name	Marke	Preis	Bestand	Favorit	Extra1	Extra2
-----	----	------	-------	-------	---------	---------	--------	--------

Extra-Spalten je Typ: Elektronik = Garantiemonate + Batterietyp, Lebensmittel = Haltbarkeitsdatum + Bio, Kleidung = Größe + Material.

Ein Detail, das beim Testen aufgefallen ist: wenn ein Name selbst ein Trennzeichen enthält (z.B. "Müller;Söhne" im Text-Format), muss das escaped werden, sonst zerschießt der Split die Zeile.

`escapeField` / `unescapeField` kümmern sich darum - wichtig ist die Reihenfolge: zuerst Backslash escapen, dann das Trennzeichen, sonst stimmt das Unescapen am Ende nicht.

LagerService

Spring- `@Service` , hält eine gemeinsame `LagerVerwaltung` -Instanz für alle Views. Lädt beim Start aus `data/lager.csv` , oder legt beim allerersten Start ein paar Demo-Artikel an, falls die Datei noch nicht existiert. Nach jeder verändernden Aktion wird automatisch wieder gespeichert. Die eigentliche Logik bleibt komplett in `LagerVerwaltung` - der Service ist nur die Klammer für "eine Instanz für alle" + Persistenz.

GUI (Vaadin)

Drei Seiten, Navigation über `MainLayout` :

- **ArtikelListView** (/) - Tabelle mit Suche, Mehrfach-Filter (Kategorie/Marke/Preis), sortierbaren Spalten, Favoriten-Stern und CRUD über einen Dialog (`ArtikelFormDialog`). Die Felder im Dialog ändern sich je nach gewählter Kategorie. Validiert wird nicht im Dialog selbst, sondern in den Artikel-Konstruktoren - der Dialog fängt die Exception nur und zeigt sie an.
- **StatistikView** (/statistik) - zeigt die Auswertungen aus `LagerVerwaltung` an (Gesamtwert, Durchschnitt, teuerster/günstigster Artikel, Artikel je Kategorie, ...). Reine Anzeige, keine eigene Logik.
- **VerlaufView** (/verlauf) - listet das Änderungs-Log, neueste Einträge zuerst.

Tests

`DateiServiceTest` prüft vor allem den Roundtrip (speichern -> laden -> gleiche Werte), für beide Formate, plus die Sonderzeichen-Fälle (Semikolon, Komma, Backslash im Namen) und ein paar Fehlerfälle (leere Datei, nicht existente Datei).

`LagerVerwaltungTest` deckt die `aendern` -Methode ab: normaler Ersatz-Fall, unbekannte ID, `null` als Eingabe, und ID-Mismatch zwischen Parameter und neuem Artikel.

Starten

```
cd app
.\mvnw spring-boot:run
```

Erster Start dauert ein paar Minuten wegen dem Frontend-Build, danach läuft die App auf `http://localhost:8080` . Laufzeit-Daten liegen in `app/data/lager.csv` (wird automatisch angelegt, ist in

`.gitignore`).

Tests: `cd app` und dann `.\mvnw test` .

Für die reine Backend-Logik ohne Browser gibt's `LagerverwaltungDemo.main()` zum direkten Starten in der IDE - zeigt Filter, Sortierung, Auswertungen, Gruppierung, Favoriten und Datei-Roundtrip auf der Konsole durch.

Stand

Umgesetzt sind die drei GUI-Seiten mit Navigation, Laden beim Start / Speichern bei jeder Änderung über `LagerService` , JUnit-Tests für `LagerVerwaltung` (Ändern-Funktion), und die Ändern-Funktion selbst. Was man noch ausbauen könnte: mehr Tests für Filter/Sortierung/ Auswertungen, und etwas mehr Feinschliff bei der Formular-Validierung im Dialog.